CS 527 Research Project Report: Group-9

Runchu Tian, Vikas Reddy
University of Illinois at Urbana-Champaign
runchut2,vredd4@illinois.edu

ABSTRACT

Coding LLMs in agentic settings process large volumes of source code through tool outputs such as file reads and search results. This code context, passed as text tokens, can be vast—easily reaching hundreds of thousands of tokens per session. Recent "optical compression" methods suggest that rendering text as images and feeding them to a vision-capable LLM may offer a different cost–performance tradeoff. We study a focused question: without any post-training, can an agentic coding LLM read source code from images as effectively as from text? We evaluate GPT-5-mini on a 100-instance subset of SWE-bench Verified using the mini-swe-agent framework, comparing a standard text-based agent against an optical variant that renders code-heavy tool outputs as monospace images. To determine whether the model can reliably read code from images, we first conduct a pilot study sweeping rendering parameters and establish that GPT-5-mini achieves $\leq$1.5% character error rate on rendered Python code and 100% identifier recall (12pt font, 120-character width). On the main benchmark, both conditions resolve exactly 51 of 100 instances (51%), with largely overlapping but not identical success sets (41 shared, 10 unique to each). The optical condition consumes 29% more input tokens and costs approximately 1.9× more per instance. We draw two conclusions. First, optical code input without any post-training is fully viable: a vision-capable model can read and reason about code from images as accurately as from text. Second, image tokens are currently more expensive than text tokens, so realizing the cost-saving potential of optical compression will likely require post-training to reduce the number of vision tokens per image—an avenue explored by concurrent work such as DeepSeek-OCR.

1 ARTIFACT INFORMATION

GitHub Repository: `https://github.com/Rachum-thu/cs527-proj`

2 INTRODUCTION

Targeted Problem. Long-context problem is a core bottleneck for coding LLMs: realistic workflows include repository-level code snippets, long test logs, and multiple tool observations that easily exceed typical context budgets. When the full context is not provided, models can fail due to missing critical information. When the full context is provided, inference becomes costly, and the model may still be unreliable because attention is spread across redundant or noisy text; empirically, accuracy can degrade when relevant evidence appears in the middle of long contexts [1, 9, 16]. Therefore, effective long-context handling is crucial for coding LLMs [3, 25].

Existing Methods. State-of-the-art long-context strategies for coding LLMs mainly fall into three categories. First, retrieval-based approaches select small subsets of repository context (files, functions, or chunks) to fit within a budget [12, 15, 19]. Second, pruning-based methods attempt to discard irrelevant lines in logs/diffs or compress context while preserving salient parts; agentic systems such as SWE-agent operationalize iterative retrieval and tool interaction for software tasks [20, 28, 29]. Third, summarization-based methods replace long observations with model-generated summaries or compressed prompts [10]. While effective in many settings, each category has its own shortcomings (e.g., missing relevant evidence, over-pruning, or unfaithful summaries), so robust long-context handling for coding LLMs remains an open problem [9, 16].

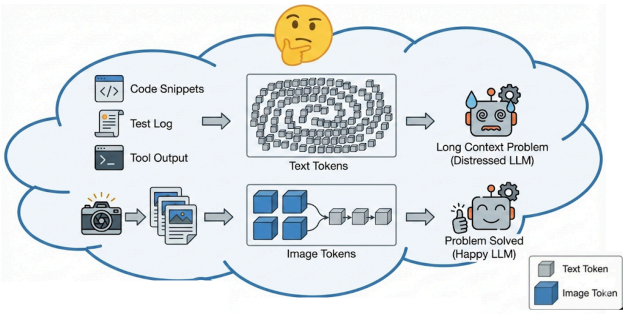

Figure 1: Overview of our main idea. Evaluating coding LLM on long context input: plain-text vs. "OCR" compression.

Research Gap. Recently, a new context compression technique, which we refer to as *optical compression*, has emerged. The key idea is to turn long text into images and feed them to a vision-capable LLM, potentially reducing the number of text tokens needed while still conveying the same information. This approach has shown promise in document-oriented settings [13, 14, 17], but its utility for coding LLMs is unclear. On the one hand, optical compression could make it cheaper to pass long tool outputs (e.g., logs, stack traces, diffs, and multi-file code) and may mitigate long-context brittleness by preserving more raw evidence [16]. On the other hand, image-based encoding can introduce new error modes (e.g., character-level corruption or missed symbols) which can be particularly harmful for code [21]. Overall, there is limited systematic evidence on whether off-the-shelf coding LLMs benefit from optical compression, and how such encoding affects the cost–fidelity–performance tradeoff [4, 26].

Research Question. To this end, we focus on a question: in an agentic coding setting with no post-training, can a vision-capable LLM read source code from rendered images as effectively as from plain text, and what are the cost and behavioral implications?

Proposed Approach. As shown in Figure 1, we evaluate optical compression as a drop-in replacement for text-based code context in an agentic coding pipeline. We build on `mini-swe-agent` and implement an `OpticalAgent` that intercepts code-heavy tool outputs (identified by a simple heuristic) and renders them as monospace images before passing them to GPT-5-mini. The issue description and non-code outputs (error messages, directory listings, short search results) remain as text. This observation-level design ensures that the bulk of code the model reads—file contents accessed via `sed`, `cat`, and similar commands—is presented as images.

Experiments. We conducted a two-phase evaluation. First, a *readability pilot study* swept rendering parameters (font sizes 8–20pt, page widths 100–140 characters) across six diverse Python code samples, establishing that GPT-5-mini achieves $\leq$1.5% character error rate and 100% identifier recall at 12pt/120-character width. Second, we ran the full agentic pipeline on a 100-instance stratified subset of SWE-bench Verified, comparing the text baseline against the optical condition. We measured resolve rate, token usage, dollar cost, wall clock time, and behavioral differences.

Results. We draw two main conclusions. First, **optical code input without any post-training is fully viable**: the vision-capable model reads and reasons about code from images as accurately as from text, achieving identical resolve rates (51/100 for both conditions), with 41 instances resolved by both and 10 unique to each. Second, **image tokens are currently more expensive than text tokens**: the optical agent consumes 29% more input tokens and costs 1.9× more per instance ($0.06 vs. $0.03). Realizing the cost-saving potential of optical compression will likely require post-training to reduce the number of vision tokens per image, as explored by concurrent work such as DeepSeek-OCR [26].

Contributions. The main contributions of this work are:

- A readability pilot study establishing that GPT-5-mini can reliably read Python source code from rendered images (CER ≤1.5%, 100% identifier recall), with an analysis of error patterns including the model's tendency to semantically rewrite regex rather than transcribe it.
- A controlled evaluation demonstrating that optical code input is a viable drop-in replacement for text in an agentic coding pipeline, achieving equivalent resolve rates (51% vs. 51%) on SWE-bench Verified without any post-training.
- An analysis showing that while feasible, optical encoding is currently more expensive per instance, suggesting that post-training (e.g., learned visual encoders) is needed to make optical compression cost-competitive.

3 PROPOSED TECHNIQUE

3.1 Approach Overview

Input & Output. We study a long-context coding task in an agentic setting: given (i) a natural-language issue description and (ii) access to a software repository at a fixed snapshot, the model interacts with a shell environment over multiple turns—reading files, running tests, and editing code—to produce a patch that resolves the described issue [11]. We build on `mini-swe-agent` [28], a minimal agentic framework where the agent repeatedly queries a language model, executes bash commands, and observes the output.

Main Idea. Our core idea is to replace the *text* representation of code with an *image* representation wherever the agent encounters source code during its workflow. Concretely, when a tool command returns a code-heavy output (e.g., reading a file with `sed -n` or `cat`), we render that output as a monospace document image and present it to the vision-capable model as an image input instead of text. This lets us test whether a model can perform multi-step software engineering through the visual channel without any post-training or architectural change.

The key insight is that code reading happens predominantly during the agent loop, not in the initial problem description. Our analysis of agent traces shows that `sed`/`cat`-style file reads account for over 80% of the code tokens the model processes, while code blocks in the issue description are a small fraction. Therefore, we apply optical encoding at the *observation level*—intercepting tool outputs—rather than only at the input level.

3.2 Approach Details

Observation-Level Optical Encoding. At each step of the agent loop, the model issues a bash command, the environment executes it, and the output is returned as a message. In the standard (text) setup, this output is a plain-text string. In our optical setup, we add an intermediate step: if the output is identified as code (see below), we render it as one or more images and present those images to the model in lieu of the text.

A technical constraint shapes the implementation: the OpenAI tool-call API requires tool-response messages to contain only text, not images. We work around this by (1) replacing the code output in the tool message with a short placeholder (e.g., "`[Code output (243 lines) rendered as image in the next message]`") and (2) injecting a follow-up `user` message containing the rendered images. The model thus sees the tool acknowledgment followed immediately by the visual code content.

Table 1: Examples of the code detection heuristic. Outputs exceeding both thresholds (length > 2,000 chars, code ratio > 20%) are rendered as images; others remain as text.

Command	Chars	Code%	Action
`sed -n '1,240p' models.py`	7,859	31%	Image
`cat utils.py`	7,574	33%	Image
`python3 repro.py`	1,052	35%	Text
`grep -n "def prepare"`	87	0%	Text
`ls -la`	1,030	0%	Text
`git diff -- file.py`	1,268	6%	Text

Algorithm 1: Observation-Level Optical Encoding

Input: Tool output text t, rendering parameters θ
Output: Message sequence $\mathcal{M}$ (tool message + optional image message)

$r \leftarrow \text{code_ratio}(t)$;
if $|t| > 2000$ ***and*** $r > 0.2$ **then**
  $n \leftarrow |\text{lines}(t)|$;
  $chunks \leftarrow \textsc{SplitByLines}(t,\ \theta.max_lines)$;
  $images \leftarrow [\textsc{Render}(c, \theta) \text{ for } c \in chunks]$;
  $m_{\text{tool}} \leftarrow$ "`[Code output (`n` lines) rendered as image]`";
  $m_{\text{img}} \leftarrow \textsc{UserMessage}(images)$;
  return $(m_{\text{tool}},\ m_{\text{img}})$
return $(\textsc{ToolMessage}(t),)$; `// keep as text`

Code Detection Heuristic. Not all tool outputs should be rendered as images—directory listings, error messages, short grep results, and test outputs are better left as text. We apply a simple heuristic: an output is rendered as an image if and only if it satisfies both of the following conditions:

- **Length > 2,000 characters**, ensuring that only substantial outputs are converted (short outputs gain little from compression).
- **Code ratio > 20%**, where "code ratio" is the fraction of lines whose stripped content starts with a Python keyword or indentation pattern (`def`, `class`, `import`, `from`, `if`, `for`, `return`, `while`, `try:`, `#`, or four-space indent).

Table 1 illustrates the heuristic on representative tool outputs from our experiments. In practice, 25% of all tool outputs are rendered as images (92 out of 366 across 20 pilot instances), and these account for the vast majority of code tokens the model processes.

Rendering Pipeline. Each code output that passes the heuristic is rendered into one or more PNG images using the Python Pillow library. The rendering specification was determined through a systematic pilot study (Section 4.2): 12pt DejaVu Sans Mono, 120-character page width, 80 lines per image, black text on white background, no syntax highlighting. Files exceeding 80 lines are split across multiple images, each labeled with a header indicating the segment number.

Algorithm 1 summarizes the decision logic applied at each agent step. For outputs classified as code, the tool message contains a placeholder and a follow-up user message delivers the rendered images. For all other outputs, the original text is passed through unchanged.

```
django/db/models/fields/__init__.py
class Field(RegisterLookupMixin):
    """Base class for all field types."""

    empty_strings_allowed = True
    creation_counter = 0

    def __init__(self, verbose_name=None, name=None,
                 primary_key=False, max_length=None,
                 unique=False, blank=False, null=False,
                 db_index=False, default=NOT_PROVIDED,
                 editable=True, serialize=True,
                 help_text='', validators=(),
                 error_messages=None):
        self.name = name
        self.verbose_name = verbose_name
        self.primary_key = primary_key
        self.max_length = max_length
        self.unique = unique
        self.blank = blank
        self.null = null

    def __eq__(self, other):
        if isinstance(other, Field):
            return (self.creation_counter ==
                    other.creation_counter)
        return NotImplemented

    def __lt__(self, other):
        if isinstance(other, Field):
            return (self.creation_counter <
                    other.creation_counter)
        return NotImplemented
```

Figure 2: Example of a rendered code image as seen by the model. The agent issued a file-read command; the output was detected as code (31% code ratio, 7,859 characters) and rendered at 12pt/120-char width.

Rendering example. Figure 2 shows a representative rendered image produced by our pipeline. The image uses 12pt DejaVu Sans Mono on a white background, with a gray header bar indicating the source file. This is the exact format the model receives when a code-reading command (e.g., `sed -n '1,240p' fields/__init__.py`) returns a code-heavy output.

Integration with mini-swe-agent. We implement the optical encoding as a subclass of the default agent (`OpticalAgent`) that overrides the `execute_actions` method. The agent loop proceeds as follows: (1) the model receives the conversation history and generates a bash command via tool call; (2) the environment executes the command and returns the output; (3) the `OpticalAgent` checks whether the output passes the code detection heuristic; (4a) if yes, the output is rendered as images, the tool message is replaced with a placeholder, and a user message with the images is appended; (4b) if no, the output is passed as text normally; (5) the model sees the updated history and generates the next command. This cycle repeats until the model submits a patch or hits the step/cost limit.

This design requires no changes to the framework's core code, model interface, or environment layer. The same prompt template, decoding settings, and evaluation protocol are used for both conditions, ensuring that the input representation is the only experimental variable. The issue description (problem statement) is also preprocessed: markdown code blocks are rendered as images using the same pipeline.

4 EXPERIMENTAL EVALUATION

We aim to answer the following research questions:

- **RQ1 (Effectiveness).** Does providing code context as images affect patch success on SWE-bench Verified compared to plain-text context?
- **RQ2 (Efficiency).** How does optical encoding affect token usage, dollar cost, and wall clock time?
- **RQ3 (Fidelity).** How accurately can GPT-5-mini read Python source code from rendered images, and which code features are most error-prone?

4.1 Evaluation Setting

Benchmark and task. We evaluate on a 100-instance stratified subset of SWE-bench Verified [18], drawn from all 12 repositories in the dataset using proportional allocation (see Appendix for the full instance list). Each instance provides a repository snapshot and a GitHub issue description; the agent must produce a patch that fixes the issue [11].

Test-based evaluation. We apply each generated patch and run the benchmark's containerized test suite, following the official SWE-bench protocol: `FAIL_TO_PASS` tests must pass, and `PASS_TO_PASS` tests must remain passing [11].

Table 2: Prompt alignment pilot (20 instances). Adding an explicit "do not commit" instruction eliminates the behavioral divergence.

Condition	Prompt	Git commit	Empty patch	Avg steps
Text	before fix	2/20 (10%)	1/20 (5%)	23.4
Optical	before fix	17/20 (85%)	4/20 (20%)	20.1
Optical	after fix	0/20 (0%)	1/20 (5%)	17.9

Model and agent. We use GPT-5-mini (`gpt-5-mini-2025-08-07`) as the backbone model within the `mini-swe-agent` framework. The agent has a 50-step limit and a $3 cost limit per instance. All other settings (prompt template, tool interface, environment) are identical across conditions.

Conditions. We compare two conditions:

- **Text.** Standard setup—all tool outputs are returned as plain text.
- **Optical.** Code-heavy tool outputs (length > 2,000 characters and code ratio > 20%) are rendered as monospace images and presented via a follow-up user message; all other outputs remain as text. Issue-description code blocks are also rendered as images.

Both conditions use identical prompts except that the optical prompt includes one additional system instruction ("Some code content may be provided as images") and an explicit "do not `git commit`" directive (see Section 4.1).

Metrics. For each instance × condition, we record: resolve status (pass/-fail), total input and output tokens (summed across all agent steps), dollar cost, wall clock time (from first to last model query timestamp), number of agent steps, and—for the optical condition—the number of images rendered and the fraction of tool outputs rendered as images.

Readability confirmation. The pilot study in Section 4.2 establishes that GPT-5-mini can reliably read Python code from our rendered images: CER ≤1.5% on standard code, 100% identifier recall, and 100% indentation accuracy at the selected configuration (12pt, 120-character width). This confirms that downstream performance differences, if any, are not attributable to the model's inability to parse the images.

Prompt alignment. In an initial 20-instance pilot, we observed that image inputs caused GPT-5-mini to alter its workflow behavior: the model ran `git commit` before generating the patch in 85–90% of optical runs versus only 10% of text runs, causing `git diff` to return empty output. Table 2 shows this effect and its resolution.

After adding the instruction "Do NOT run `git add` or `git commit`; your changes must remain as uncommitted modifications," the commit behavior dropped to 0% and the empty-patch rate matched the text baseline. All main experiments use the fixed prompt. This finding is noteworthy in its own right: multimodal inputs can alter model behavior in ways entirely unrelated to the content, a confound that must be controlled in any fair comparison.

4.2 Image Readability Pilot

Before the main experiment, we conducted a pilot study to determine image rendering parameters that give the best fidelity-to-cost tradeoff. The question is: *at what font size, page width, and image height can GPT-5-mini accurately read Python code from images?*

Motivation. If the model cannot reliably recover code from images, any downstream performance difference would be uninterpretable—we would not know whether failures stem from optical compression itself or from

Table 3: Font size sweep (page width = 100 chars, 80 lines/image). CER and token cost as a function of font size.

Font	Avg CER	Id Recall	Indent Acc	Avg Tokens
f8	0.084	0.97	0.60	538
f10	0.046	0.99	0.83	700
f12	**0.023**	**0.99**	**0.99**	**1,038**
f14	0.019	0.99	0.82	1,338
f16	0.030	0.99	0.69	1,853
f18	0.018	1.00	0.83	2,161
f20	0.027	0.99	0.82	2,349

Table 4: Effect of page width and image height on fidelity (at 12pt and 14pt).

Setting	CER	Indent	Id Recall	Tokens
f12, w100	0.023	0.99	0.99	1,038
f12, w120	**0.023**	**1.00**	**1.00**	**1,160**
f14, w120	0.018	0.83	1.00	1,524
f14, w140	0.015	0.83	1.00	1,762
f12, l120	0.025	0.83	0.99	1,009
f14, l120	0.021	0.74	0.99	1,306

poor rendering choices. This section establishes a lower bound on image readability before proceeding to the main evaluation.

Test samples. We analyzed the SWE-bench Verified dataset to understand code length distributions: the median patch is 22 lines and the 90th percentile is 77 lines. Guided by this, we selected six Python source code samples (46–155 lines) covering import-heavy modules, nested control flow with exception handling, and regex/special-character-heavy code.

Metrics. For each rendering configuration, we prompted GPT-5-mini to transcribe the rendered image and compared the transcription against the ground truth using: character error rate (CER), identifier recall (fraction of Python names recovered), and indentation accuracy (fraction of matching lines with correct leading whitespace).

Round 1: Font size sweep. We tested seven font sizes (8–20pt) at a fixed 100-character page width. Table 3 shows the results.

CER drops sharply from f8 (8.4%) to f12 (2.3%), indicating a threshold below which the vision encoder cannot resolve monospace characters. Above f14, CER does not consistently improve, while tokens scale linearly. Identifier recall remains $\geq$0.97 at all sizes. We selected **12pt**: CER within 0.4 percentage points of the best (18pt), at 52% fewer tokens.

Round 2: Page width and image height. Error analysis revealed that the dominant error at f12/w100 was *line truncation*: lines exceeding 100 characters wrapped in the image, and the model reproduced the break. We tested wider pages (120, 140 chars) and taller images (120 lines). Table 4 shows the results.

Widening to 120 characters eliminated truncation for >97% of SWE-bench code lines and pushed indentation accuracy to 1.00 at f12. Further widening to 140 chars yielded marginal improvement at 52% higher cost. Increasing image height from 80 to 120 lines *worsened* quality: indentation accuracy dropped from 0.99 to 0.83, consistent with a "lost at the bottom" effect where the vision encoder loses fidelity in tall images.

Error patterns. Two residual error modes persist at the selected configuration (f12, w120, 80 lines/image).

Table 5: Regex rewriting: the model generates semantically similar but syntactically different expressions. Differences are underlined.

Ground truth	Transcription
`@(\w+(?:\.\w+)*(?:\([^)]*\))?)\s*$`	`@(\w+(\.\w+)*)\s*(\([^)]*\))?\s*$`
`^(?:from\s+([\w.]+)\s+)?import\s+(.+)$`	`^\s*(?:from\s+[\w\.]+\s+)?import\s+[\w\*,\s]+`

Table 6: Per-sample CER at f12/w120. Identifier recall is 1.00 for all samples.

Sample	Feature	Lines	CER
dense_imports	imports, dataclass	46	0.1%
agent_short	class, type hints	50	1.5%
special_chars	regex, escapes	58	9.4%
complex_logic	nested logic, f-str	96	1.1%
agent_medium	class, URLs	100	1.0%
agent_full	full module	155	0.9%

Regex rewriting. The model does not transcribe complex regex patterns character-by-character; instead, it parses the pattern semantically and regenerates what it considers an equivalent expression. Table 5 shows three examples from our test suite. In each case, the transcribed regex captures the same language class as the original but uses different grouping or quantifier syntax. This behavior inflates CER on regex-heavy code (~9% vs. ~1% on standard code) but does not necessarily indicate a failure of understanding—the model demonstrably comprehends the pattern's intent.

Header injection. When content spans multiple images, the model occasionally inserts the header from the next image (e.g., "`filename.py (part 2/2)`") into the transcription output. This is a predictable, strippable artifact.

Table 6 shows per-sample CER at the final configuration.

Selected configuration. We fix **12pt font, 120-character page width, 80 lines per image** for all subsequent experiments. This achieves CER $\leq$1.5% (excluding the regex outlier) with perfect indentation accuracy and identifier recall, at 1,160 tokens per sample.

4.3 Main Results

Table 7 presents the main results on 100 SWE-bench Verified instances.

RQ1: Effectiveness. Both conditions resolve exactly 51 of 100 instances, yielding identical 51% resolve rates. However, the resolved sets are not identical. Table 8 breaks down the overlap.

The two conditions agree on 80% of instances, with 10 instances uniquely solved by each. This symmetric disagreement pattern suggests that optical encoding does not systematically help or hurt: the model is equally capable of reasoning from either modality, but introduces different variance in each.

Disagreement analysis. We examined the 20 disagreement instances for systematic patterns. The optical-only set is dominated by Django instances (7/10), compared to only 3/10 in the text-only set. The text-only set includes more instances from repositories with complex mathematical notation (Sympy: 3/10, xarray: 3/10). One hypothesis is that Django's highly standardized code style (consistent indentation, naming conventions, docstring patterns) is particularly well-suited to visual parsing, while mathematical/scientific code with dense symbolic expressions may lose precision

Table 7: Main results: Text vs. Optical on 100 SWE-bench Verified instances.

Metric	Text	Optical
Instances resolved	51/100 (51%)	51/100 (51%)
Patches with content	96/100	98/100
Limits exceeded	0	2
Avg input tokens	292,060	376,722
Avg output tokens	5,143	5,596
Avg images per instance	—	14.5
Avg agent steps	21.3	19.8
Avg cost per instance	$0.03	$0.06
Total cost (100 inst.)	$3.25	$6.25
Avg wall clock (s)	120.8	165.0

Table 8: Overlap analysis: instance-level agreement between Text and Optical.

Category	Count
Both resolved (agree)	41
Neither resolved (agree)	39
Text only (Text > Optical)	10
Optical only (Optical > Text)	10
Total agreement	80/100 (80%)

in image rendering. However, the small sample size (10 per group) limits the statistical power of this comparison.

We also note that GPT-5-mini does not support `temperature=0`, so all outputs involve sampling noise. Some of the 20 disagreement instances may simply reflect run-to-run variance rather than genuine modality effects. Disentangling these factors would require multiple runs per instance per condition, which we leave to future work.

RQ2: Efficiency. The optical condition is more expensive on every efficiency metric. Input tokens increase by 29% (376K vs. 292K) because image tokens are less dense than text tokens—the same 240 lines of code that occupy ∼2,000 text tokens require ∼3,500 image tokens. Dollar cost nearly doubles ($0.06 vs. $0.03), reflecting both the higher token count and the typically higher per-token price of image inputs. Wall clock time increases by 37% (165s vs. 121s), attributable to image rendering overhead and larger payloads.

However, the optical agent takes *fewer* steps on average (19.8 vs. 21.3). One possible explanation is that code images provide a more visually salient overview of file structure, allowing the model to locate relevant sections faster. We note that this step reduction is not large enough to offset the per-step token increase.

These results highlight an important distinction: optical encoding is *feasible* (equivalent task performance) but not yet *cheaper*. The cost overhead stems from the current vision tokenization scheme, where image tokens are less dense than text tokens. Realizing actual cost savings would require post-training to compress vision token counts—for example, DeepSeek-OCR [26] achieves 10× compression ratios with a trained encoder, which could potentially make optical encoding cost-competitive or even cheaper than text.

RQ3: Fidelity and code detection. In the optical condition, 25% of tool outputs are classified as code and rendered as images (92/366 in the 20-instance pilot). On the full 100-instance run, the optical agent rendered an average of 14.5 images per instance. The code detection heuristic (length > 2,000 chars and code ratio > 20%) produces very few false positives: only 3 out of 92 rendered outputs in the pilot were from non-standard commands (Python script executions whose output happened to contain code-like patterns), and all three were arguably appropriate to render.

The pilot study (Section 4.2) established that CER on standard Python code is ≤1.5% with 100% identifier recall. The main experiment confirms that this fidelity level is sufficient: the 51% resolve rate matches the text baseline, indicating that any character-level errors introduced by optical encoding do not materially affect the model's ability to understand code and generate correct patches.

Case studies. We examine three instances to illustrate the qualitative differences between conditions.

Optical succeeds, text fails (`django-13401`*):* The issue concerns Django's `Field` class ordering behavior. Both agents read the same source file (`fields/__init__.py`), but followed different strategies. The optical agent (14 steps) performed targeted searches for `__eq__`/`__hash__`/`__lt__` methods and ran verification tests before submitting, producing a correct patch. The text agent (11 steps) applied a patch more quickly but skipped edge-case verification, resulting in a patch that failed the test suite. This suggests that the visual layout may have helped the optical agent perceive the method structure more holistically, leading to more thorough verification.

Text succeeds, optical fails (`django-11532`*):* The issue involves a change in Django's database backend. The text agent (12 steps) identified and fixed the issue correctly. The optical agent (16 steps) read the same code and produced a patch of similar length, but the fix was subtly incorrect—the generated patch did not fully address the test requirements. This illustrates that while optical encoding preserves code readability, the model's reasoning from visual input can occasionally diverge, leading to different (and sometimes wrong) solution paths.

Both succeed, different paths (`astropy-14995`*):* Both agents resolved this Astropy issue, but the optical agent did so in 13 steps compared to the text agent's 21 steps—a 38% reduction. Inspecting the trajectories, the text agent explored several unrelated files before locating the correct one, while the optical agent navigated more directly. The optical agent processed 7 code images (versus raw text), yet arrived at the correct fix faster. This case illustrates the "fewer steps" trend observed in the aggregate data (19.8 vs. 21.3 average steps) and suggests that the spatial layout of code images may sometimes aid navigation efficiency, even though the overall token cost is higher.

5 RELATED WORK

Long-context handling for coding LLMs. Repository-level coding benchmarks such as SWE-bench cast software engineering as generating a patch for a real GitHub issue given a full codebase snapshot and an execution-based test oracle [11, 18]. Agentic systems typically mitigate the resulting long-context burden through three complementary strategies. First, *retrieval*: systems such as SWE-agent [28], Agentless [27], and AutoCodeRover [31] search for relevant files, functions, or code snippets and present only these to the model, avoiding the need to pass the entire repository as context. Second, *pruning and selection*: agents discard low-utility lines, constrain editable regions, or truncate long tool outputs to fit within context budgets [20, 29]. Third, *summarization and compression*: methods like LLMLingua replace long observations with model-generated summaries or compressed prompts to reduce token counts [10]. Retrieval-augmented generation is also a standard paradigm for selecting supporting context under a budget [12, 15, 19], and in code-centric settings, repository-level retrieval and representation learning are commonly used to identify relevant files or snippets [5, 7, 8, 30]. Orthogonally, recent work on extending context windows through positional encoding modifications (e.g., RoPE scaling [24]) and efficient attention mechanisms [3] has pushed context limits to 128K+

tokens, but these approaches do not reduce the per-token cost of processing long inputs. Our work explores a different axis: changing the *modality* of long code context from text to images.

Optical compression via image encoding. A separate line of work explores representing long textual contexts through the visual channel: text is rendered into a 2D layout and consumed as an image so that a vision-capable model can decode or reason over it with a smaller (or differently priced) input representation [13, 14, 17, 21]. DeepSeek-OCR explicitly frames this as *contexts optical compression*, training a specialized encoder to keep the number of vision tokens manageable under high-resolution inputs while retaining decodability [26]. AgentOCR applies optical representations to compress agent interaction histories, showing that rendering prior tool outputs as images can reduce context length without significantly degrading task performance [4]. In the document understanding domain, models such as Nougat [2] and Pix2Struct [14] are trained end-to-end to parse rendered document images, achieving strong results on scientific papers and web pages. Importantly, most reported gains in this space rely on specialized training or tailored visual encoders, leaving open how much benefit can be obtained *without* post-training when applying optical encoding to code-centric artifacts [4, 26].

Vision-capable LLMs for code tasks. Vision-language models such as GPT-4V [17] and its successors have demonstrated the ability to read and reason about code from screenshots, UI mockups, and handwritten notes. Several studies have benchmarked these models on code-from-image tasks: for example, generating code from UI screenshots [23] or understanding code structure from rendered images [21]. However, these evaluations typically focus on the model's vision capability as a *standalone skill*—asking the model to read and reproduce code from an image in a single turn. Our work differs in that we use vision as an *input channel within a multi-step agentic workflow*: the model must not only read code from images but also reason about it, plan edits, and generate patches over many turns of interaction. This places different demands on the vision encoder, requiring sustained fidelity across dozens of images per episode rather than one-shot transcription accuracy.

Agentic coding and observation formatting. The design of the agent-computer interface (ACI) has been shown to significantly affect agentic coding performance. SWE-agent [28] introduced the concept of ACI design, demonstrating that the format in which tool outputs are presented to the model—including truncation, line numbering, and context windowing—can substantially change resolve rates. The ReAct framework [29] established the interleaved reasoning-and-acting paradigm that most coding agents follow, while Reflexion [22] showed that feeding back execution observations in structured formats improves self-correction. Our work contributes to this line by demonstrating that the *modality* of observation formatting (text vs. image) is another axis of ACI design, and that changing this modality can alter model behavior in unexpected ways (Section 4.1), even when the underlying information content is preserved.

Multimodal inputs and model behavior. An emerging body of work studies how the presence of multimodal inputs affects LLM behavior beyond the content of those inputs. Studies have shown that including images in prompts can alter the model's tone, verbosity, and instruction-following behavior, even when the images are irrelevant to the task [6]. Our finding that image inputs cause GPT-5-mini to systematically change its git workflow (committing before diffing, Section 4.1) is consistent with this phenomenon: the presence of visual content appears to shift the model's behavioral distribution in ways that are independent of the image content itself. This suggests that multimodal agentic systems require careful prompt alignment to control for modality-induced behavioral confounds, a consideration that has received limited attention in the agentic coding literature.

Table 9: Comparison with related approaches.

	Post-train	Code	Agentic	Zero-shot
LLMLingua [10]	No	No	No	Yes
DeepSeek-OCR [26]	Yes	No	No	No
AgentOCR [4]	Yes	No	Yes	No
Nougat [2]	Yes	No	No	No
Agentless [27]	No	Yes	No	Yes
Ours	No	Yes	Yes	Yes

How our study differs. Rather than proposing a new trained compressor, we provide a controlled evaluation of optical encoding as a *pure input representation change* for agentic coding. We test a fixed vision-capable model (GPT-5-mini) on SWE-bench Verified and compare text against optical input at the observation level, where the agent reads code through tool outputs. Unlike DeepSeek-OCR [26] and AgentOCR [4], we require no post-training; unlike document understanding models [2, 14], we evaluate on a downstream software engineering task rather than transcription accuracy alone. Our study is, to our knowledge, the first to evaluate optical compression for code in an agentic software engineering setting with a full execution-based evaluation.

6 THREATS TO VALIDITY

Internal validity. Our code detection heuristic (code ratio $> 20\%$, length $> 2{,}000$ characters) is a simple rule that may produce false positives (rendering non-code outputs as images) or false negatives (missing code outputs). In our 20-instance pilot, only 3 of 92 rendered outputs were borderline cases (Python script outputs), and manual inspection confirmed they contained code-like content. A more sophisticated classifier could improve precision, but would introduce its own complexity and potential confounds.

The image rendering pipeline introduces design choices—font size, page width, line wrapping—that were fixed via the pilot study. A different specification might yield different results. We mitigate this by selecting parameters that achieve near-perfect fidelity ($\leq$1.5% CER, 100% identifier recall) and documenting the selection process transparently.

GPT-5-mini does not support `temperature=0`, so all outputs involve sampling noise. The 10/10 symmetric disagreement between conditions may partly reflect this randomness rather than genuine modality differences. Multiple runs per instance would be needed to disentangle these effects.

External validity. We evaluate a single model (GPT-5-mini) on a single benchmark (100-instance subset of SWE-bench Verified). Our findings may not generalize to other vision-capable LLMs with different vision encoders or to other coding tasks beyond patch generation. SWE-bench Verified is predominantly Python; optical encoding may behave differently for other languages with different visual density or indentation conventions.

Construct validity. The SWE-bench pass rate is binary: a patch either passes all required tests or fails. This does not capture partial correctness or near-miss patches. Our cost comparison depends on current OpenAI API pricing, which may change. The code detection heuristic's thresholds (2,000 chars, 20% code ratio) were chosen empirically and not optimized; different thresholds could shift the balance of what gets rendered.

7 CONCLUSION

We evaluated optical compression—rendering source code as monospace images—as a drop-in replacement for text-based code input in an agentic software engineering pipeline. On a 100-instance subset of SWE-bench Verified using GPT-5-mini, the optical condition matched the text baseline

exactly (51% resolve rate each), with 80% instance-level agreement and a symmetric 10/10 split of unique successes.

We draw two main conclusions. First, **optical code input without post-training is fully viable**: GPT-5-mini's vision encoder can read and reason about Python source code from rendered images with the same effectiveness as from text. Our readability pilot confirms this at the character level ($\leq$1.5% CER, 100% identifier recall), and the main experiment confirms it at the task level (identical resolve rates). This result is encouraging for the broader vision of optical compression: the fundamental capability is already present in current vision-capable models.

Second, **image tokens are currently more expensive than text tokens**, so optical encoding does not yet deliver cost savings. The optical agent consumed 29% more input tokens and cost 1.9× more per instance ($0.06 vs. $0.03). This overhead arises because current vision tokenizers produce more tokens per unit of information than text tokenizers. Bridging this gap will likely require post-training to compress vision token counts—concurrent work such as DeepSeek-OCR [26] achieves 10× compression ratios with a trained encoder, which could make optical encoding not only feasible but cost-competitive.

Contributions.

- A readability pilot study establishing GPT-5-mini's code-reading fidelity from images, including the discovery that the model semantically rewrites regex patterns rather than transcribing them.
- A controlled comparison demonstrating that optical code input achieves equivalent resolve rates to text in an agentic setting without any post-training, establishing feasibility.
- An analysis showing that while feasible, optical encoding is currently more expensive, motivating future work on trained visual encoders to achieve cost-competitive optical compression.

Future work. The most impactful next step is to combine our zero-shot optical encoding with a trained visual encoder (e.g., DeepSeek-OCR [26] or AgentOCR [4]) that reduces vision token counts, potentially achieving both the feasibility demonstrated here and actual cost savings. Other directions include: evaluating on additional models (Gemini, Claude) to test generalizability; hybrid approaches that render only long outputs as images while keeping short outputs as text; multiple runs per instance to disentangle modality effects from sampling noise; and extending to other languages and task types (code completion, test generation).

REFERENCES

[1] Yushi Bai, Xin Lv, Jiajie Zhang, Hongchang Lyu, Jiankai Tang, Zhidian Huang, Zhengxiao Du, Xiao Liu, Aohan Zeng, Lei Hou, Yuxiao Dong, Jie Tang, and Juanzi Li. 2023. LongBench: A Bilingual, Multitask Benchmark for Long Context Understanding. *arXiv preprint arXiv:2308.14508* (2023).

[2] Lukas Blecher, Guillem Cucurull, Thomas Scialom, and Robert Stojnic. 2023. Nougat: Neural Optical Understanding for Academic Documents. *arXiv preprint arXiv:2308.13418* (2023).

[3] Tri Dao, Daniel Y. Fu, Stefano Ermon, Atri Rudra, and Christopher Ré. 2022. FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness. *arXiv preprint arXiv:2205.14135* (2022).

[4] Lang Feng, Fuchao Yang, Feng Chen, Xin Cheng, Haiyang Xu, Zhenglin Wan, Ming Yan, and Bo An. 2026. AgentOCR: Reimagining Agent History via Optical Self-Compression. *arXiv preprint arXiv:2601.04786* (2026).

[5] Zhangyin Feng, Daya Guo, Duyu Tang, Nan Duan, Xiaocheng Feng, Ming Gong, Linjun Shou, Bing Qin, Ting Liu, Daxin Jiang, and Ming Zhou. 2020. CodeBERT: A Pre-Trained Model for Programming and Natural Languages. *arXiv preprint arXiv:2002.08155* (2020).

[6] Yichen Gong, Delong Ran, Jinyuan Liu, Conglei Wang, Tianyu Cong, Anyu Wang, Simin Duan, and Xiaoyun Wang. 2023. FigStep: Jailbreaking Large Vision-language Models via Typographic Visual Prompts. *arXiv preprint arXiv:2311.05608* (2023).

[7] Daya Guo, Shuai Lu, Nan Duan, Yanlin Wang, Ming Zhou, and Jian Yin. 2022. UniXcoder: Unified Cross-Modal Pre-training for Code Representation. *arXiv preprint arXiv:2203.03850* (2022).

[8] Daya Guo, Shuo Ren, Shuai Lu, Zhangyin Feng, Duyu Tang, Shujie Liu, Long Zhou, Nan Duan, Alexey Svyatkovskiy, Shengyu Fu, et al. 2020. GraphCodeBERT: Pre-training Code Representations with Data Flow. *arXiv preprint arXiv:2009.08366* (2020).

[9] Cheng-Ping Hsieh, Simeng Sun, Samuel Kriman, Shantanu Acharya, Dima Rekesh, Fei Jia, Yang Zhang, and Boris Ginsburg. 2024. RULER: What's the Real Context Size of Your Long-Context Language Models? *arXiv preprint arXiv:2404.06654* (2024).

[10] Huiqiang Jiang, Qianhui Wu, Chin-Yew Lin, Yuqing Yang, and Lili Qiu. 2023. LLMLingua: Compressing Prompts for Accelerated Inference of Large Language Models. *arXiv preprint arXiv:2310.05736* (2023).

[11] Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. 2023. SWE-bench: Can Language Models Resolve Real-World GitHub Issues? *arXiv preprint arXiv:2310.06770* (2023).

[12] Vladimir Karpukhin, Barlas Oguz, Sewon Min, Patrick Lewis, Ledell Wu, Sergey Edunov, Danqi Chen, and Wen-tau Yih. 2020. Dense Passage Retrieval for Open-Domain Question Answering. *arXiv preprint arXiv:2004.04906* (2020).

[13] Geewook Kim, Teakgyu Hong, Moonbin Yim, Jeongyeon Nam, Jinyoung Park, Jinyeong Yim, Wonseok Hwang, Sangdoo Yun, Dongyoon Han, and Seunghyun Park. 2021. OCR-free Document Understanding Transformer. *arXiv preprint arXiv:2111.15664* (2021).

[14] Kenton Lee, Mandar Joshi, Iulia Turc, Hexiang Hu, Fangyu Liu, Julian Eisenschlos, Urvashi Khandelwal, Peter Shaw, Ming-Wei Chang, and Kristina Toutanova. 2022. Pix2Struct: Screenshot Parsing as Pretraining for Visual Language Understanding. *arXiv preprint arXiv:2210.03347* (2022).

[15] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. 2020. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. *arXiv preprint arXiv:2005.11401* (2020).

[16] Nelson F. Liu, Kevin Lin, John Hewitt, Ashwin Paranjape, Michele Bevilacqua, Fabio Petroni, and Percy Liang. 2023. Lost in the Middle: How Language Models Use Long Contexts. *arXiv preprint arXiv:2307.03172* (2023).

[17] OpenAI. 2023. GPT-4V(ision) System Card. Technical report.

[18] OpenAI. 2024. Introducing SWE-bench Verified. OpenAI Blog. Updated February 24, 2025.

[19] Stephen Robertson and Hugo Zaragoza. 2009. The Probabilistic Relevance Framework: BM25 and Beyond. *Foundations and Trends in Information Retrieval* 3, 4 (2009), 333–389. https://doi.org/10.1561/1500000019

[20] Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu, Maria Lomeli, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. 2023. Toolformer: Language Models Can Teach Themselves to Use Tools. *arXiv preprint arXiv:2302.04761* (2023).

[21] Yongxin Shi, Dezhi Peng, Wenhui Liao, Zening Lin, Xinhong Chen, Chongyu Liu, Yuyi Zhang, and Lianwen Jin. 2023. Exploring OCR Capabilities of GPT-4V(ision): A Quantitative and In-depth Evaluation. *arXiv preprint arXiv:2310.16809* (2023).

[22] Noah Shinn, Federico Cassano, Edward Berman, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. 2023. Reflexion: Language Agents with Verbal Reinforcement Learning. *arXiv preprint arXiv:2303.11366* (2023).

[23] Chenglei Si, Yanzhe Li, Ziang Wang, Ruoxi Liu, Ziniu Jiang, Xinyi Rao, et al. 2024. Design2Code: How Far Are We From Automating Front-End Engineering? *arXiv preprint arXiv:2403.03163* (2024).

[24] Jianlin Su, Murtadha Ahmed, Yu Lu, Shengfeng Pan, Wen Bo, and Yunfeng Liu. 2024. RoFormer: Enhanced Transformer with Rotary Position Embedding. *Neurocomputing* 568 (2024), 127063. arXiv:2104.09864.

[25] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez,
Lukasz Kaiser, and Illia Polosukhin. 2017. Attention Is All You Need. In *Advances in Neural Information Processing Systems (NeurIPS).*

[26] Haoran Wei, Yaofeng Sun, and Yukun Li. 2025. DeepSeek-OCR: Contexts Optical Compression. *arXiv preprint arXiv:2510.18234* (2025).

[27] Chunqiu Steven Xia, Yinlin Deng, Soren Dunn, and Lingming Zhang. 2024. Agentless: Demystifying LLM-based Software Engineering Agents. *arXiv preprint arXiv:2407.01489* (2024).

[28] John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. 2024. SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering. *arXiv preprint arXiv:2405.15793* (2024).

[29] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. 2022. ReAct: Synergizing Reasoning and Acting in Language Models. *arXiv preprint arXiv:2210.03629* (2022).

[30] Fengji Zhang, Bei Chen, Yue Zhang, Jacky Keung, Jin Liu, Daoguang Zan, Yi Mao, Jian-Guang Lou, and Weizhu Chen. 2023. RepoCoder: Repository-Level Code Completion Through Iterative Retrieval and Generation. *arXiv preprint arXiv:2303.12570* (2023).

[31] Yuntong Zhang, Haifeng Ruan, Zhiyu Fan, and Abhik Roychoudhury. 2024. AutoCodeRover: Autonomous Program Improvement. *arXiv preprint arXiv:2404.05427* (2024).

A TRANSCRIPTION PROMPT

The following prompt was used in the image readability pilot (Section 4.2) to evaluate GPT-5-mini's code-reading fidelity:

```
Transcribe the exact text content shown in this
image. Preserve all whitespace, indentation, symbols,
and formatting exactly as shown. Do not add any
explanation or commentary --- output ONLY the transcribed
text.
```

B INSTANCE SELECTION

The 100-instance subset was constructed from SWE-bench Verified using proportional stratified sampling across all 12 repositories (seed=42). Table 10 shows the allocation. The full list of instance IDs and the selection script (`scripts/build_subset_100.py`) are available in the artifact repository.

Table 10: Repository distribution of the 100-instance subset.

Repository	N	Repository	N
django	44	pydata/xarray	4
sympy	15	pytest-dev	4
sphinx-doc	9	psf/requests	2
matplotlib	7	pylint-dev	2
scikit-learn	6	mwaskom/seaborn	1
astropy	5	pallets/flask	1